

TravelMate Bildlade-Flow Systemanalyse

Stand: Analyse des aktuellen Repositories ohne Codeänderung.

Ziel: den gesamten Bildlade-Flow als System untersuchen und nicht von nur einem Fehler ausgehen. Sichtbares Symptom: echte Aktivitätsbilder können kurz sichtbar sein und danach wieder durch Platzhalter ersetzt werden.

Kurzfazit

Es ist sehr wahrscheinlich nicht ein einzelner Bug. Der aktuelle Flow hat mindestens drei starke Problemstellen: ActivityGallery hält echte Bilder lokal und verliert sie beim Remount; replaceTrip(updated) kann Bilder aus dem Client-State wieder entfernen; Galerie und Preloader laden parallel denselben Backend-Endpunkt, während Backend-Enrichment bei Race Conditions leer zurückgeben kann.

Die sichere Richtung ist: Bilddaten zentralisieren, Trip-Merges bildsicher machen und Backend-Enrichment idempotent gegen Parallelität absichern. Erst danach lohnt Feinschliff an Placeholdern oder CSS.

Systemueberblick

Trip wird geöffnet über `/trips/{id}`. `TripDto` enthält `ActivityDto.images`. Diese Daten landen in `useTripWorkspace.trip`. `DayOrbitCard` wählt `selectedActivity`. `ActivityGallery` baut daraus `displaySlides`. Wenn `activity.images` oder lokale `enrichedImages` echte URLs enthalten, werden Real-Slides gebaut; sonst ein Kategorie-Platzhalter.

Parallel dazu lädt `ActivityGallery` selbst `POST /activities/{id}/images`. Zusätzlich lädt `useTripWorkspace.preloadUpcomingImages` denselben Endpoint für den aktiven Tag und zwei Folgetage. Nur der Workspace schreibt zentral in `trip.value`; `ActivityGallery` speichert eigene Ergebnisse lokal in `enrichedImages`.

1. Datenebene / Backend

Echte Bildreferenzen werden in `activity_images` gespeichert. `ActivityImageEntity` hat `activity`, `source`, `providerRef`, `url`, `alt`, `credit` und `sortOrder`. Die Tabelle hat eine Unique Constraint auf `activity_id`, `source` und `provider_ref`. Das verhindert echte Duplikate auf DB-Ebene.

```
@Table(name = "activity_images", uniqueConstraints = @UniqueConstraint(columnNames
= {"activity_id", "source", "provider_ref"}))
public class ActivityImageEntity {
    public Long id;
    public ActivityEntity activity;
    public ActivitySource source;
    public String providerRef;
    public String url;
    public String alt;
    public String credit;
    public int sortOrder;
}
```

`POST /activities/{id}/images` ist grundsätzlich idempotent, wenn Bilder bereits existieren. `GooglePlacesImageService.ensureImages` gibt dann `activity.images` zurück. Aber parallele Requests sind nicht aktiv gesperrt. Wenn zwei Requests gleichzeitig starten und beide keine Bilder sehen, kann einer beim Persistieren gegen die Unique Constraint laufen und im `Catch List.of()` zurückgeben.

2. API-/DTO-Ebene

ActivityImageDto enthaelt aktuell nur url, alt, credit und source. Es fehlen id, activityId, imageId, providerRef, mediaUrl und eine explizite Kennzeichnung, ob es ein echtes Bild oder ein Placeholder ist.

```
public record ActivityImageDto(
    String url,
    String alt,
    String credit,
    String source
) {}
```

Initialer Trip-Load und nachtraegliches Enrichment nutzen beide ActivityImageDto.from(...). Das Format ist daher grundsatzlich gleich. Die Identitaet des Bildes wird im Frontend aber nur aus der URL abgeleitet. Bei Google-Bildern ist diese URL meist /activities/{activityId}/images/{imageId}/media.

3. Frontend-State / Trip-Store

Der zentrale Trip-State liegt in frontend/composables/useTripWorkspace.ts. Trip wird via loadTrip geladen. Spaetere Aktionen ersetzen den ganzen Trip mit replaceTrip(updated).

```
const replaceTrip = (updated: Trip) => {
    trip.value = updated
    trips.value = trips.value.map(item => item.id === updated.id ? updated : item)
}
```

Das ist gefaehrlich, weil vorhandene clientseitig gemergte Bilder nicht erhalten werden, wenn eine spaetere API-Antwort dieselbe Activity mit images: [] liefert. Der Workspace hat zwar updateActivityImages, das leere Listen ignoriert, aber replaceTrip umgeht diesen Schutz vollstaendig.

```
const updateActivityImages = (activityId: number, images: ActivityImage[]) => {
    if (!trip.value) return
    if (!images.length) return
    for (const day of trip.value.days) {
        for (const item of day.activities) {
            if (item.activity.id === activityId) {
                item.activity.images = mergeImages(item.activity.images, images)
            }
        }
    }
}
```

4. Galerie-Logik

ActivityGallery sammelt propImages aus props.activity.activity.images und localImages aus enrichedImages. propImages stammen aus dem zentralen Trip-State, localImages aus einem lokalen Galerie-Request.

```
const propImages = computed(() =>
    Array.isArray(props.activity?.activity?.images) ?
    props.activity?.activity?.images : [])
const localImages = computed(() => Array.isArray(enrichedImages.value) ?
    enrichedImages.value : [])
```

Die finale Entscheidung ist simpel: Wenn `reallImages.length` groesser 0 ist, baut `displaySlides` echte Slides. Wenn `reallImages` leer ist, baut `displaySlides` einen Fallback-Slide. Dadurch kann ein Placeholder wieder aktiver Slide werden, sobald echte Bilder aus dem State verschwinden.

5. Load-/Error-State

`ActivityGallery` verwaltet `failedImageUrls`, `loadedImageUrls` und `imageLoadStatus` lokal. Keys sind URLs, nicht image IDs. Innerhalb derselben Instanz gewinnt `loaded` gegen `failed`, denn `markImageFailed` ignoriert URLs, die bereits in `loadedImageUrls` liegen. Nach einem Remount ist dieser Schutz weg.

```
const markImageFailed = (url: string) => {
  if (loadedImageUrls.value.includes(url)) return
  if (!failedImageUrls.value.includes(url)) {
    failedImageUrls.value = [...failedImageUrls.value, url]
  }
}
```

Beim Activity-Wechsel wird lokaler Bildstatus geloescht: `enrichedImages`, `failedImageUrls`, `loadedImageUrls` und `imageLoadStatus`. Da die Galerie ueber den `selectedActivity`-Key neu gemountet wird, geht lokaler Erfolg schnell verloren.

6. Hintergrund-Preloader

Der Preloader laedt denselben Endpoint wie die Galerie: `POST /activities/{activityId}/images`. Er schreibt nur in den Trip-State und nicht direkt in den Galerie-Fehlerstatus. Der Preloader-Status wird von der Galerie nicht fuer die Anzeigeentscheidung genutzt.

Trotzdem kann der Preloader indirekt Probleme verstaerken: Er erhoehrt die Zahl paralleler Requests; er kann Activities fuer 120 Sekunden als `failed` markieren; und spaetere Trip-Replacements koennen Bilder wieder verlieren.

7. Media-Proxy / Redirect

Das Frontend bekommt meist keine direkte Google-URL, sondern eine interne Proxy-URL. `ActivityResource.imageMedia` ruft `GooglePlacesImageService.resolvePhotoUri` auf und gibt einen temporaeren Redirect zur echten Google Photo URL zurueck.

Moegliche Fehler: 403 durch Key-Restriction, Timeout, ungueltige Google Photo Referenz, 500/503 im Backend oder langsamer Redirect. Ein sichtbarer `img`-Fehler landet in `failedImageUrls`. Ein Preloader-Fehler landet nur im Preloader-Status.

8. CSS / Layering

Fallback und echtes Bild liegen gleichzeitig im DOM. Beide sind absolut positioniert. Der Fallback hat `z-index 1`, das echte Bild `z-index 2` und `opacity 1`. Daher sieht CSS im aktuellen Stand nicht nach der Hauptursache aus. Der Placeholder wird sichtbar, wenn das echte Bild nicht mehr im DOM ist oder als `failed` gilt.

```
.gallery-fallback-image { z-index: 1; opacity: 1; }
.gallery-real-image { z-index: 2; opacity: 1; transition: opacity 120ms ease; }
```

9. Lifecycle / Vue Watcher

Es gibt keine React Effects, sondern Vue Watcher. Der wichtigste Watcher sitzt in ActivityGallery und setzt beim Activity-Wechsel den Bildstatus zurueck. DayOrbitCard mountet die Galerie zudem ueber :key="selectedActivity.id" neu. pages/trips/[id].vue startet den Preloader beim Mount mindestens doppelt: einmal ueber syncDayFromRoute und danach nochmal direkt.

Ursachenmatrix

1. Lokale Galerie-Bilder gehen beim Remount verloren. Wahrscheinlichkeit: sehr hoch. Beleg: ActivityGallery speichert POST-Ergebnis nur in enrichedImages; Watcher setzt enrichedImages auf null. Gegenbeleg: Wenn Preloader vorher zentral gemerged hat, bleiben Bilder erhalten. Dateien: ActivityGallery.vue, DayOrbitCard.vue. Test: Log bei Mount proplImages/locallImages/reallImages. Fix: Galerie-Ergebnisse zentral mergen oder Galerie-POST entfernen. Risiko: mittel.
2. replaceTrip(updated) ueberschreibt Bilder mit aelteren oder leeren Daten. Wahrscheinlichkeit: hoch. Beleg: trip.value = updated ohne Merge-Schutz. Gegenbeleg: Backend-DTO enthaelt Bilder, wenn DB sie kennt. Dateien: useTripWorkspace.ts, useTripPlanner.ts. Test: Log oldImages/newImages in replaceTrip. Fix: vorhandene echte Bilder behalten, wenn neue Antwort images leer hat. Risiko: niedrig bis mittel.
3. Galerie und Preloader laden parallel denselben Endpoint. Wahrscheinlichkeit: hoch. Beleg: Beide rufen POST /activities/{id}/images. Gegenbeleg: Backend ist idempotent, wenn Bilder existieren. Dateien: ActivityGallery.vue, useTripWorkspace.ts, GooglePlacesImageService.java. Test: Request-Logs mit Quelle Gallery/Preloader. Fix: einen Besitzer definieren oder shared in-flight Registry. Risiko: niedrig.
4. Backend-Enrichment ist nicht ausreichend race-condition-resistent. Wahrscheinlichkeit: mittel-hoch. Beleg: Unique Constraint existiert, Catch gibt aber List.of() zurueck. Gegenbeleg: DB verhindert echte Duplikate. Dateien: GooglePlacesImageService.java, ActivityImageEntity.java. Test: zwei parallele POSTs auf dieselbe Activity. Fix: nach Fehler DB requery und vorhandene Bilder zurueckgeben; optional Lock pro Activity. Risiko: mittel.
5. Media-Proxy-Fehler markiert Bild lokal als failed. Wahrscheinlichkeit: mittel. Beleg: img @error schreibt failedImageUrls; Proxy loest Google URL jedes Mal neu auf. Gegenbeleg: loaded gewinnt innerhalb derselben Instanz. Dateien: ActivityGallery.vue, ActivityResource.java, GooglePlacesImageService.java. Test: Media-Proxy-Status und onError loggen. Fix: failed als temporaer behandeln; loaded stabiler speichern. Risiko: niedrig.
6. DTO hat keine stabile Bild-ID. Wahrscheinlichkeit: mittel. Beleg: ActivityImageDto enthaelt nur url, alt, credit, source. Gegenbeleg: imageld steckt in der Proxy-URL. Dateien: ActivityImageDto.java, frontend/types/trip.ts. Test: URL-Keys ueber Reload und Remount vergleichen. Fix: DTO um id, activityId und mediaUrl erweitern. Risiko: mittel wegen API-Vertrag.
7. Preloader markiert Activity failed und retryt 120 Sekunden nicht. Wahrscheinlichkeit: mittel. Beleg: imagePreloadFailedAt plus imageRetryAfterMs 120000. Gegenbeleg: Galerie kann trotzdem selbst laden. Datei: useTripWorkspace.ts. Test: failedAt und spaetere Galerie-Anzeige vergleichen. Fix: Retry flexibler und sichtbare Activity priorisieren. Risiko: niedrig.
8. CSS versteckt echte Bilder. Wahrscheinlichkeit: niedrig im aktuellen Stand. Beleg: beide Bilder liegen absolut. Gegenbeleg: real image hat z-index 2 und opacity 1. Dateien: polish.css, ActivityGallery.vue. Test: DOM und computed style pruefen. Fix: real image immer ueber fallback, kein opacity-gate. Risiko: niedrig.

9. Backend-Matching findet fuer viele Activities keine Fotos. Wahrscheinlichkeit: hoch fuer viele Platzhalter, aber niedrig fuer kurz sichtbar dann weg. Beleg: findGooglePlace kann Optional.empty liefern. Gegenbeleg: wenn echte Bilder kurz sichtbar waren, gab es Bilder. Datei: GooglePlacesImageService.java. Test: Match-Score, reject reason, photos count loggen. Fix: Matching verbessern. Risiko: falsche Fotos moeglich.

Top 3 Ursachen

1. Lokale Galerie-Bilder sind nicht dauerhaft. Das passt am besten zum Symptom. 2. Ungeschuetztes replaceTrip(updated) kann Bilder wieder verlieren. 3. Doppelte Enrichment-Requests plus Backend-Race Conditions verstaerken das Problem.

Sichere Fix-Reihenfolge

Schritt 1: temporaere Diagnose-Logs einfuegen. Log bei Galerie-Mount, POST-Ergebnis, onLoad, onError, displaySlides, updateActivityImages und replaceTrip. Backend: ensureImages Start/Ende, vorhandene Bilder, Match, gespeicherte Bilder, Exception mit DB-Requery.

Schritt 2: zentralen Bildbesitz festlegen. Empfehlung: useTripWorkspace ist Quelle der Wahrheit. ActivityGallery zeigt nur activity.images oder merged seine Ergebnisse zentral zurueck.

Schritt 3: replaceTrip bildsicher machen. Eine leere oder fehlende images-Liste aus einer spaeteren Trip-Antwort darf vorhandene echte Bilder nicht entfernen.

Schritt 4: Backend-Enrichment race-resistent machen. Bei Fehlern nach dem Speichern oder bei Unique Constraint danach DB erneut lesen und vorhandene Bilder zurueckgeben.

Schritt 5: DTO stabilisieren. ActivityImageDto sollte id, activityId, mediaUrl, source und optional providerRef enthalten. Platzhalter bleiben reine Frontend-Fallbacks.

Schritt 6: Galerie robuster machen. Wenn echte Bilder vorhanden sind, baut die Galerie keinen fallback-only-Slide. loaded gewinnt immer gegen failed. failed ist temporaer, nicht permanent.

Zielzustand

Echte Bilder haben Prioritaet vor Platzhaltern. Platzhalter sind nur visuelles Sicherheitsnetz. Geladene echte Bilder werden nicht nachtraeglich degradiert. Aeltere API- oder State-Antworten duerfen neuere echte Bilder nicht entfernen. Preloader darf die Galerie nicht kaputtsteuern. Backend-Enrichment ist idempotent und race-condition-resistent.